



# Orientação a Objetos

Diagramas de Sequência (UML)

**Prof. Edson José Pacheco**

2026

## Fundamentos

- ▶ O papel do diagrama de sequência na UML
- ▶ Modelagem de interações temporais
- ▶ Relação com casos de uso e classes
- ▶ Visão dinâmica do sistema

## Elementos e Notação

- ▶ Objetos e linhas de vida
- ▶ Mensagens síncronas e assíncronas
- ▶ Barras de ativação e retornos
- ▶ Fragmentos combinados (alt, loop, opt)

## Padrões de Interação

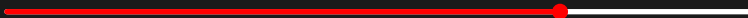
- ▶ Chamada síncrona e resposta
- ▶ Comunicação assíncrona (MQTT, eventos)
- ▶ Criação e destruição de objetos
- ▶ Auto-mensagem e delegação

## Prática e Mapeamento

- ▶ Traduzir sequências para código Python
- ▶ Exemplos em SCADA e automação industrial
- ▶ Comunicação com dispositivos de campo
- ▶ Exercício prático com sistema supervisório



CLIQUE PARA ABRIR NO NAVEGADOR



 Um *Overview* sobre Diagramas de Sequencia

O Diagrama de Sequência modela a **interação entre objetos ao longo do tempo**, exibindo a **ordem temporal** das mensagens trocadas para realizar uma funcionalidade.

## Características

- ▶ Integrante dos diagramas de interação da UML
- ▶ Eixo vertical: fluxo do **tempo**
- ▶ Eixo horizontal: **objetos** participantes
- ▶ Detalha o comportamento dinâmico

## Aplicações

- ▶ Detalhar a realização de casos de uso
- ▶ Documentar protocolos de comunicação
- ▶ Modelar interação com hardware
- ▶ Validar fluxos de erro e exceção

**O diagrama responde à pergunta:** *“Em que ordem os objetos se comunicam para realizar uma tarefa?”*

## Cenário: Comando de Disjuntor

O caso de uso “Comandar disjuntor” envolve múltiplos componentes:

- ▶ IHM (interface do operador)
- ▶ Módulo de autenticação
- ▶ Controlador SCADA
- ▶ Gateway de comunicação
- ▶ IED/Relé de proteção (campo)

**Pergunta:** Em que ordem eles interagem? Quem espera quem? O que acontece se o IED não responder?

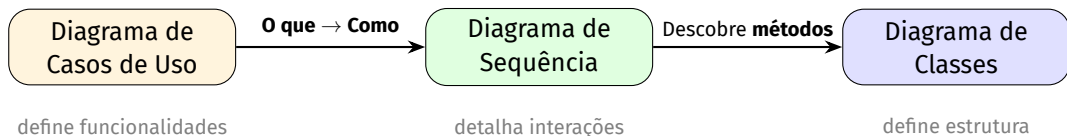
## Com Diagrama de Sequência

O diagrama mostra **passo a passo**:

1. Operador solicita abertura na IHM
2. IHM envia credenciais ao módulo de autenticação
3. Módulo valida e retorna token
4. IHM envia comando ao controlador SCADA
5. SCADA encaminha ao gateway
6. Gateway envia comando IEC 61850 ao IED
7. IED confirma execução

O diagrama de sequência torna **explícita** a dependência temporal entre componentes de software e hardware.

O diagrama de sequência detalha o “como” dos casos de uso:



**Caso de Uso:** “Comandar disjuntor” inclui “Autenticar operador”

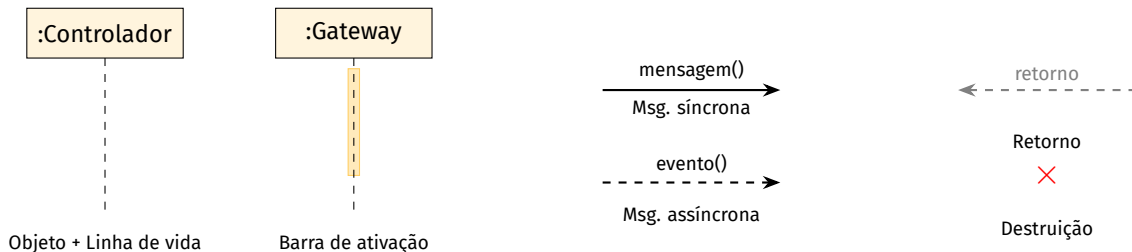
**Sequência:** Mostra a ordem exata de chamadas entre IHM, Auth, SCADA e IED para realizar o comando

Cada caso de uso pode gerar **um ou mais** diagramas de sequência (cenário principal + cenários alternativos).

# Elementos e Notação

---





- ▶ **Objeto:** Instância participante da interação.
- ▶ **Linha de vida:** Representa a existência temporal do objeto.
- ▶ **Ativação:** Indica o período de processamento do objeto.
- ▶ **Mensagem:** Seta indicando a comunicação entre instâncias.



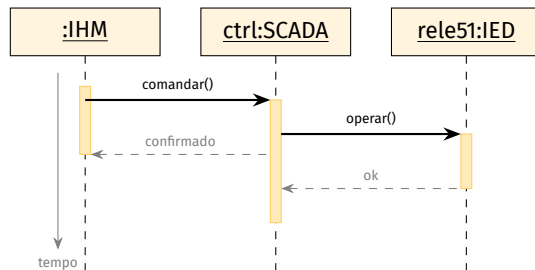
**Objeto:** Instância de uma classe que participa da interação, posicionada no topo do diagrama.

## Formatos de nomeação:

- ▶ :Classe — objeto anônimo
- ▶ nome:Classe — objeto nomeado
- ▶ nome — sem classe explícita

## Exemplos em Automação:

- ▶ :IHM, rele51:IED, gw01:Gateway



**Mensagens** representam a comunicação entre objetos. Existem quatro tipos principais:

| Tipo              | Notação         | Semântica                      | Exemplo EE         |
|-------------------|-----------------|--------------------------------|--------------------|
| <b>Síncrona</b>   | Seta cheia →    | Emissor <b>espera</b> resposta | Leitura Modbus RTU |
| <b>Assíncrona</b> | Seta aberta --> | Emissor <b>não espera</b>      | Publicação MQTT    |
| <b>Retorno</b>    | Tracejada ←--   | Resultado da chamada           | Valor de medição   |
| <b>Criação</b>    | Seta até objeto | Instancia novo objeto          | Sessão de comando  |

**Analogia EE:** Mensagem **síncrona** = protocolo *request/response* (Modbus, HTTP). Mensagem **assíncrona** = protocolo *publish/subscribe* (MQTT, eventos).

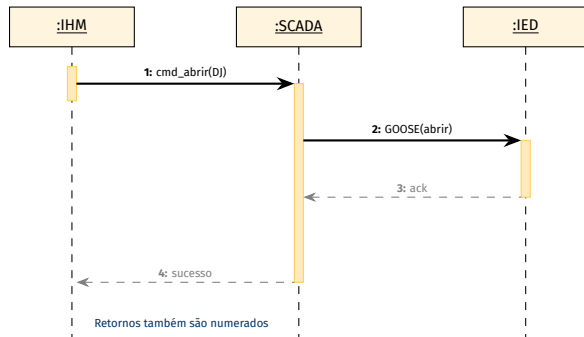
**Mensagens podem ser numeradas. Quando o são, sempre são numeradas sequencialmente** para facilitar a rastreabilidade entre diagrama, descrição textual e código.

## Numeração simples (mais comum):

- ▶ 1, 2, 3, 4...na ordem temporal
- ▶ Suficiente para a maioria dos diagramas
- ▶ Facilita referência no código: # passo 4

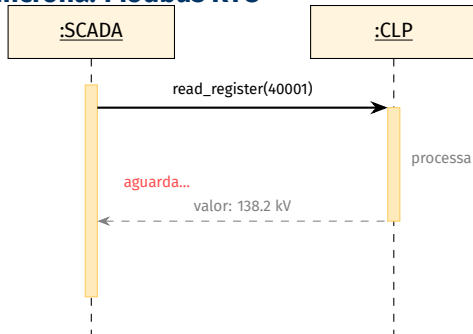
## Numeração hierárquica:

- ▶ 1 → 1.1, 1.2 (sub-chamadas)
- ▶ Útil quando uma mensagem dispara várias outras
- ▶ Ex: 6: enviar\_61850() dispara 6.1: montar\_goose() e 6.2: send()



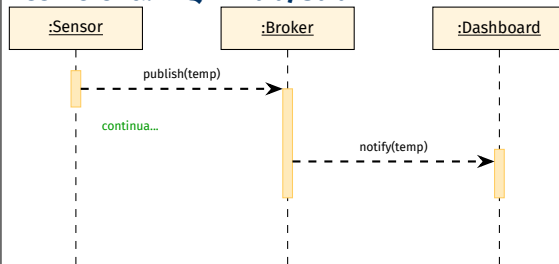
**Dica:** A numeração permite mapear cada passo para o código: o comentário # 2: GOOSE(abrir) no Python referencia diretamente a mensagem 2 do diagrama.

## Síncrona: Modbus RTU



O SCADA **bloqueia** a execução até receber a resposta. Se houver falha, ocorre *timeout*.

## Assíncrona: MQTT Pub/Sub



O sensor publica os dados e **segue o processamento** imediatamente, sem aguardar os assinantes.

## O tipo de mensagem define a implementação em código:

| UML        | Python                                  |
|------------|-----------------------------------------|
| Síncrona   | chamada de método                       |
| Assíncrona | callback / publish                      |
| Retorno    | return                                  |
| Criação    | <code>__init__()</code>                 |
| Destruição | <code>del</code> / <code>close()</code> |

Python

```
valor = clp.read_register(40001) # sincrona

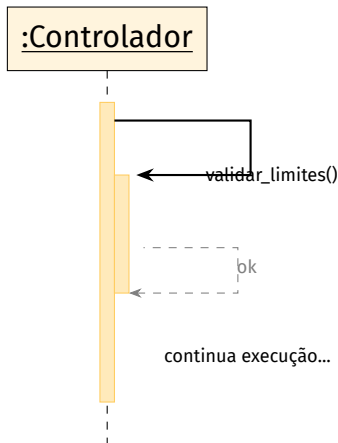
# Mensagem assincrona = publish (fire & forget)
client.publish("scada/temp", payload)

# Retorno = return do metodo
def read_register(self, addr):
    # ... processamento
    return valor

# Criacao = instanciacao
sessao = SessaoComando(disjuntor)

# Destruiacao = encerramento
sessao.close()
```

Um objeto pode enviar mensagem para si mesmo, representando processamento interno.



Python

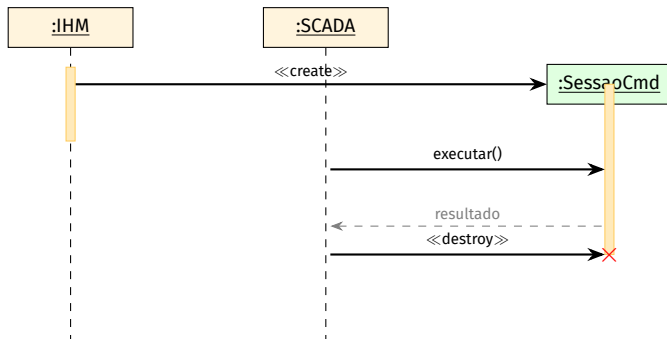
```
class ControladorSCADA:
    def comandar_disjuntor(self, tag, acao):
        # auto-mensagem: validacao interna
        self._validar_limites(tag, acao)

        # continua execucao...
        return self.gateway.enviar(tag, acao)

    def _validar_limites(self, tag, acao):
        """Auto-mensagem: metodo privado"""
        if acao == "fechar":
            status = self.ler_status(tag)
            if status.bloqueio_ativo:
                raise BloqueioError("...")
```

A auto-mensagem mapeia para chamada de **método privado** (`self._metodo()`), criando uma segunda barra de ativação sobreposta.

Objetos podem ser instanciados ou removidos dinamicamente durante a execução.



- ▶ **Criação:** A seta `<<create>>` atinge o corpo do objeto, indicando sua instanciação.
- ▶ **Destruição:** A linha de vida é encerrada por um × após a mensagem `<<destroy>>`.
- ▶ **Python:** Mapeado para `obj = Classe()` e encerramento via `del` ou métodos `close()`.

# Fragmentos Combinados

---



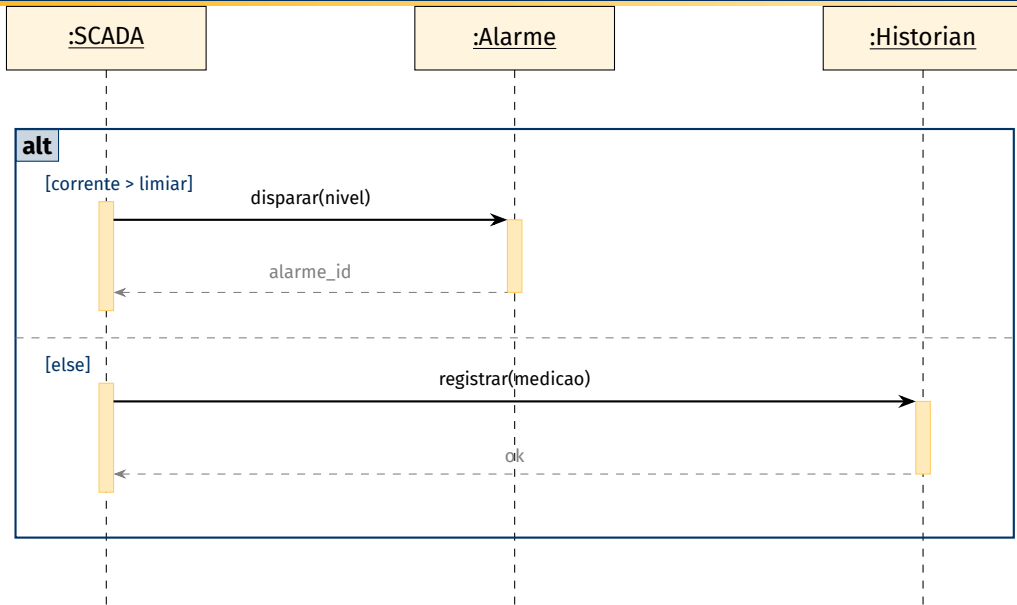


**Fragmentos combinados** modelam **estruturas de controle** no diagrama de sequência, como condicionais, laços e paralelismo.

| Fragmento    | Semântica             | Exemplo EE                    | Python       |
|--------------|-----------------------|-------------------------------|--------------|
| <b>alt</b>   | Condicional (if/else) | Se tensão normal / sobrecarga | if/elif/else |
| <b>opt</b>   | Opcional (if simples) | Se flag de log ativo          | if cond:     |
| <b>loop</b>  | Repetição             | Polling de sensores           | while/for    |
| <b>break</b> | Saída antecipada      | Timeout de comunicação        | break/raise  |
| <b>par</b>   | Execução paralela     | Ler múltiplos IEDs            | threading    |

**Nota visual:** O nome do fragmento é exibido em um **pentágono** no canto superior esquerdo do quadro que envolve as mensagens.

# Fragmento alt — Condicional

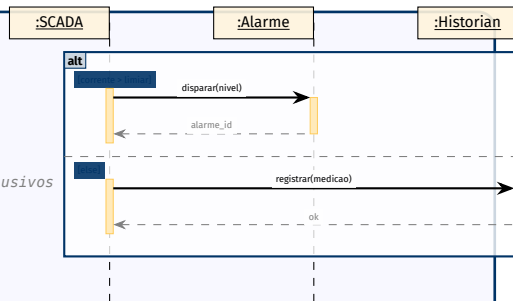


```
class MonitoramentoSCADA:
    def __init__(self, alarme, historian, limiar):
        self.alarme = alarme
        self.historian = historian
        self.limiar = limiar

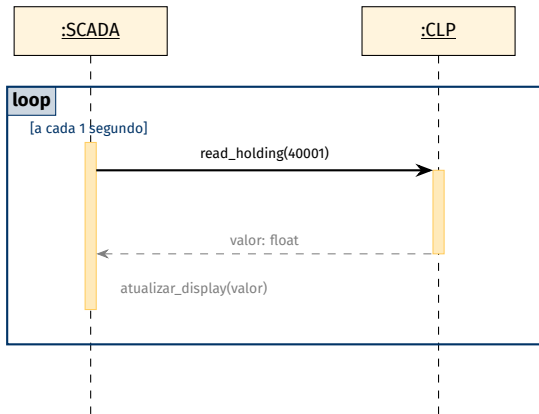
    def processar_medicao(self, medicao):
        # O fragmento 'alt' define caminhos mutuamente exclusivos
        # O fluxo percorrido depende da condicao de guarda

        if medicao.corrente > self.limiar:
            # Caminho [corrente > limiar]
            alarme_id = self.alarme.disparar(medicao.nivel)
            return alarme_id
        else:
            # Caminho [else]
            status = self.historian.registrar(medicao)
            return status

# A estrutura if/else em Python implementa a logica de decisao.
# O diagrama correspondente esta no canto superior direito.
```



**Nota:** A condição de guarda do diagrama UML mapeia-se diretamente para a expressão lógica do `if`. O bloco `else` representa o caminho alternativo obrigatório definido no fragmento.

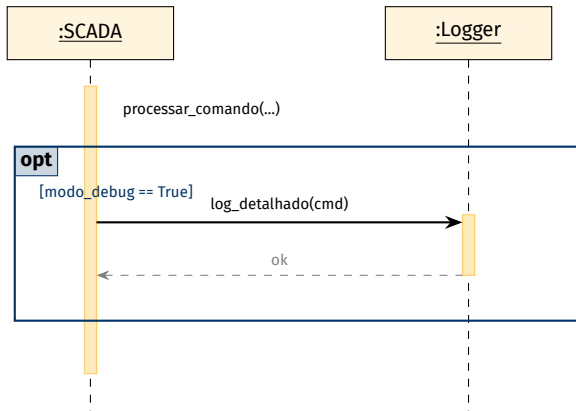


Python

```
import time

class PollingService:
    def monitorar_continuo(self, clp, endereco):
        # fragmento loop [a cada 1 segundo]
        while self.ativo:
            valor = clp.read_holding(endereco)
            self.atualizar_display(valor)
            time.sleep(1)
```

Representa o **polling** periódico. Variações incluem: loop [1..n], loop [para cada item] ou loop [enquanto ativo].



Python

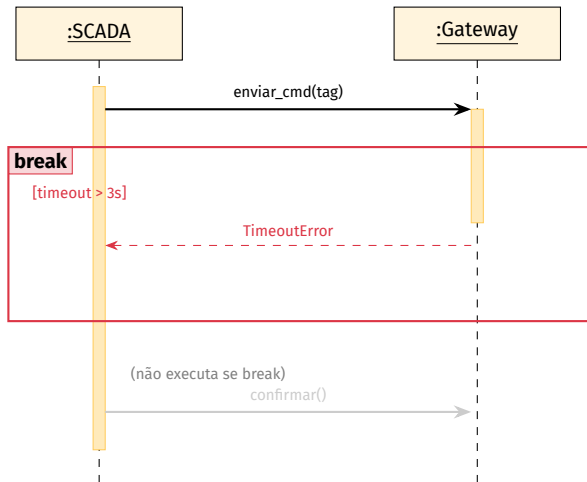
```
class ControladorSCADA:
    def processar_comando(self, cmd):
        resultado = self._executar(cmd)

        # fragmento opt [modo_debug]
        if self.modo_debug:
            self.logger.log_detalhado(
                cmd)

        return resultado
```

O **opt** equivale a um **alt sem else**. Mapeia-se para a relação de *extend* nos Casos de Uso: o comportamento é condicional e facultativo.

O **fragmento break** interrompe a sequência quando uma **condição excepcional** é satisfeita.



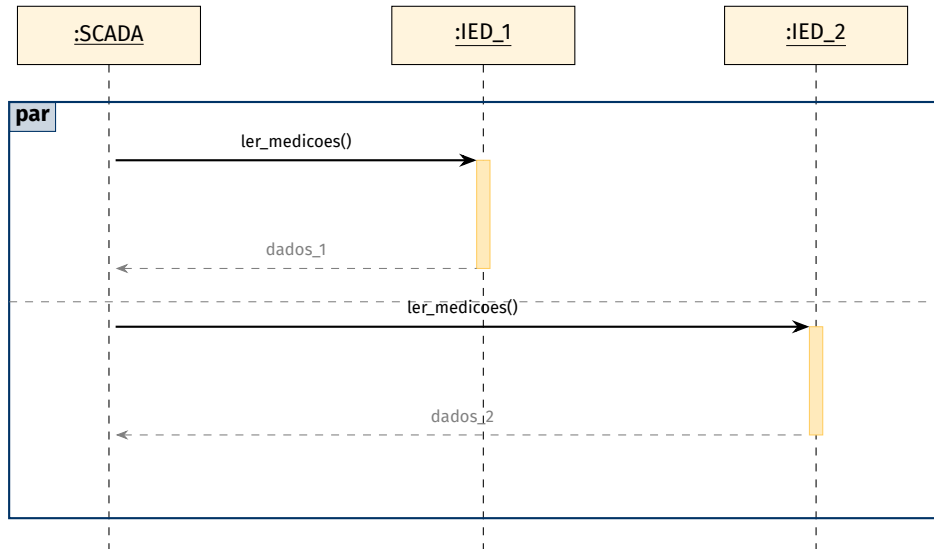
Python

```
class ControladorSCADA:
    def comandar Equipamento(self, tag, cmd):
        try:
            resp = self.gateway.enviar_cmd(tag)
        except TimeoutError:
            # fragmento break [timeout > 3s]
            self.registrar_falha(tag, "timeout")
            raise # interrompe a sequencia

        # so executa se nao houve break
        self.confirmar(tag, resp)
```

O **break** é essencial em sistemas de tempo real. Em comunicação com dispositivos de campo, *timeouts* são frequentes e devem ser tratados explicitamente.

O fragmento par modela operações que ocorrem de forma concorrente ou simultânea.

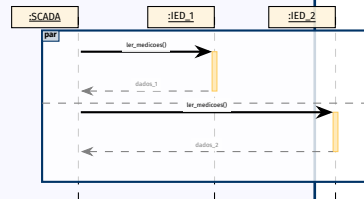


```
from concurrent.futures import ThreadPoolExecutor

class ColetorSCADA:
    def ler_todos_ieds(self, lista_ieds):
        # O fragmento 'par' indica execucao multithread ou assincrona
        # No Python, utilizamos o ThreadPoolExecutor para concorrência
        with ThreadPoolExecutor() as pool:
            # Dispara as chamadas de leitura simultaneamente
            futuros = {
                pool.submit(ied.ler_medicoes): ied
                for ied in lista_ieds
            }

            resultados = {}
            for f in futuros:
                ied = futuros[f]
                # Coleta os resultados conforme as threads finalizam
                resultados[ied.id] = f.result()

        return resultados
```



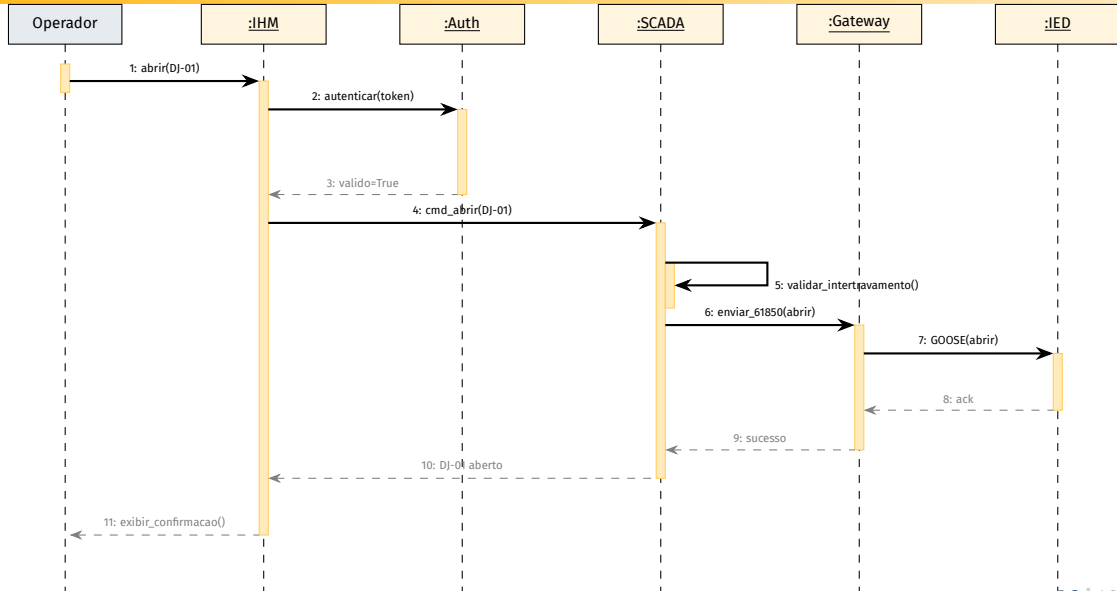


# Exemplos Completos

---



# Exemplo 1: Comando de Disjuntor (SCADA)



# Exemplo 1: Mapeamento para Python

```
class IHM:
    def abrir_disjuntor(self, operador, tag_dj):
        # 2-3: autenticar (<<include>> do caso de uso)
        if not self.auth.autenticar(operador.token):
            raise PermissionError("Token invalido")

        # 4: enviar comando ao controlador SCADA
        resultado = self.scada.cmd_abrir(tag_dj)

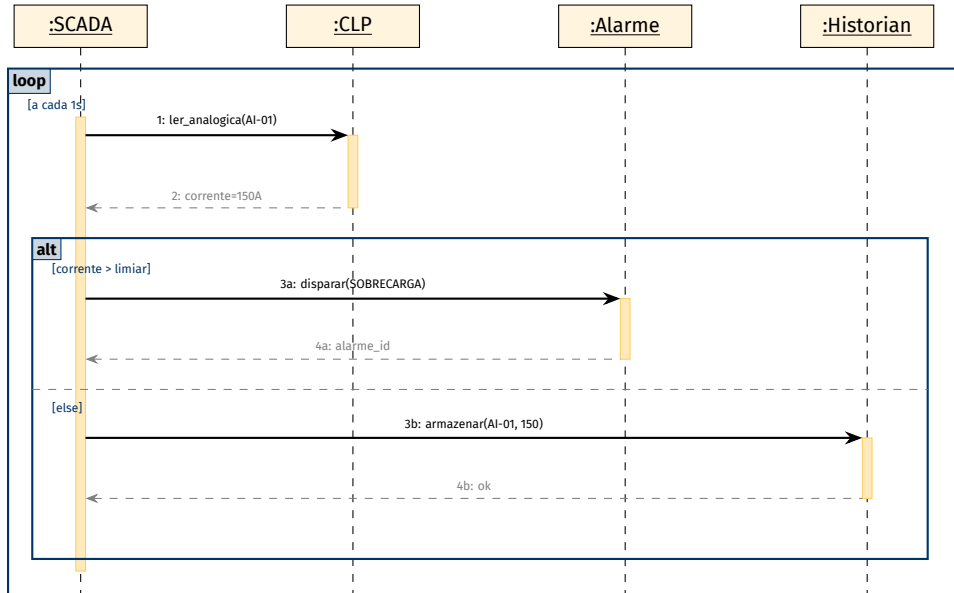
        # 11: exibir confirmacao na tela
        self.exibir_confirmacao(tag_dj, resultado)

class ControladorSCADA:
    def cmd_abrir(self, tag_dj):
        # 5: auto-mensagem (validacao interna)
        self._validar_intertravamento(tag_dj)

        # 6-9: enviar via gateway IEC 61850
        resp = self.gateway.enviar_61850("abrir", tag_dj)
        return resp # 10: retorno para IHM

class GatewayIEC61850:
    def enviar_61850(self, acao, tag):
        # 7-8: comunicacao GOOSE com IED
        frame = self._montar_goose(acao, tag)
        ack = self.ied_client.send(frame, timeout=3)
        return "sucesso" if ack else "falha"
```

# Exemplo 2: Monitoramento com Alarme



# Exemplo 2: Mapeamento para Python (SCADA)

Python

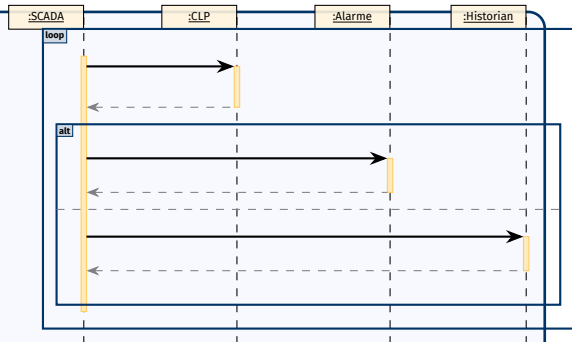
```
import time

class MonitoramentoSCADA:
    def __init__(self, clp, alarme, historian, limiar):
        self.clp = clp
        self.alarme = alarme
        self.historian = historian
        self.limiar = limiar

    def ciclo_monitoramento(self, tag_ai):
        # fragmento loop [a cada 1s]
        while self.ativo:
            # 1-2: leitura sincrona do CLP
            corrente = self.clp.ler_analogica(tag_ai)

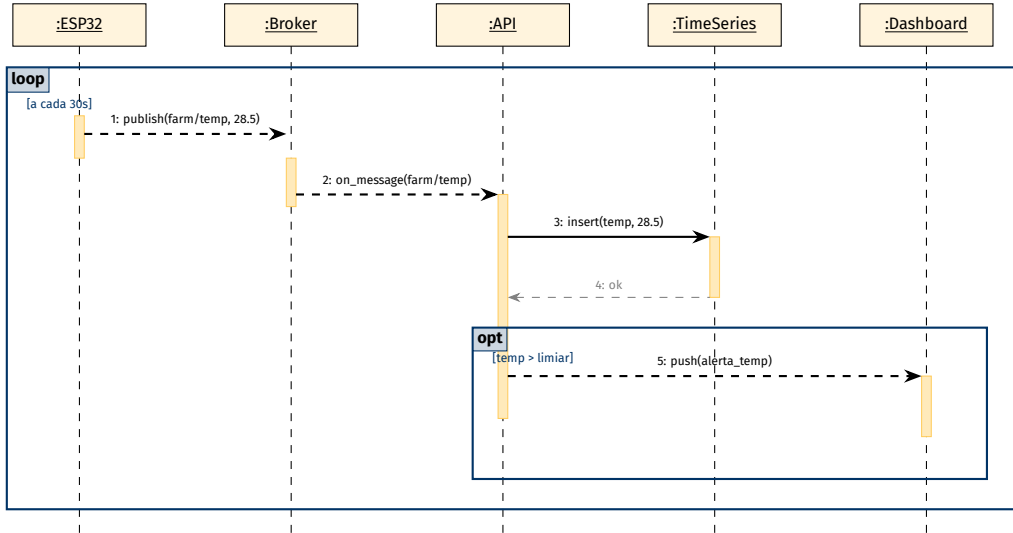
            # fragmento alt [corrente > limiar]
            if corrente > self.limiar:
                # 3a-4a: disparar alarme
                aid = self.alarme.disparar("SOBRECARGA")
            else:
                # 3b-4b: armazenar no historian
                self.historian.armazenar(tag_ai, corrente)

            time.sleep(1) # intervalo do loop
```



**Nota:** O bloco while implementa o fragmento loop, enquanto o if/else garante a exclusividade mútua do fragmento alt.

# Exemplo 3: IoT — Sensor via MQTT



# Exemplo 3: Mapeamento IoT (MQTT)

Python

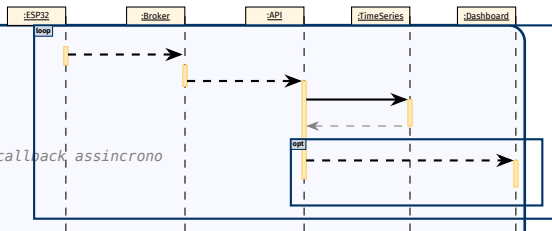
```
import paho.mqtt.client as mqtt
import json

class APIBackend:
    """Subscriber MQTT no servidor"""
    def on_message(self, client, userdata, msg): # 2: callback assincrono
        dados = json.loads(msg.payload)

        # 3-4: armazenamento sincrono em banco de dados
        self.timeseries.insert(msg.topic, dados["v"])

        # fragmento opt [temp > limiar]
        if dados["v"] > self.limiar:
            # 5: notificacao via push
            self.dashboard.push(f"ALERTA: temp={dados['v']}")

# 0 modelo combina eventos assincronos (MQTT) com chamadas sincronas (DB).
```



**Semântica:** A mensagem tracejada no diagrama indica que a `on_message` é disparada por um evento externo, sem bloqueio do Broker.

# Exercício Prático

---





**Desafio:** Desenvolva o diagrama de sequência para o cenário de atualização de parâmetros.

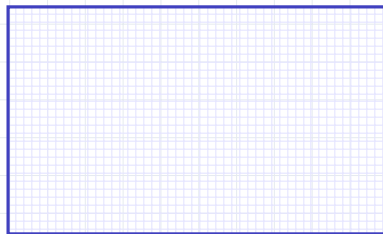
## UC “Configurar Relé” — Cenário Principal

O Engenheiro de Proteção atualiza os ajustes de um relé digital via supervisório:

1. Eng. solicita configuração do relé R51 na IHM.
2. IHM solicita autenticação ao módulo Auth (*include* obrigatório).
3. Auth valida credenciais e retorna o *token*.
4. IHM encaminha os novos parâmetros ao Controlador SCADA.
5. SCADA realiza a validação interna dos parâmetros (auto-mensagem).
6. SCADA transmite a configuração ao IED via protocolo IEC 61850.
7. **[alt]** Se o IED confirma: SCADA registra no Historian.  
Se o IED rejeita: SCADA sinaliza a falha para a IHM.
8. IHM apresenta o resultado final ao Engenheiro.



## Espaço da Resolução



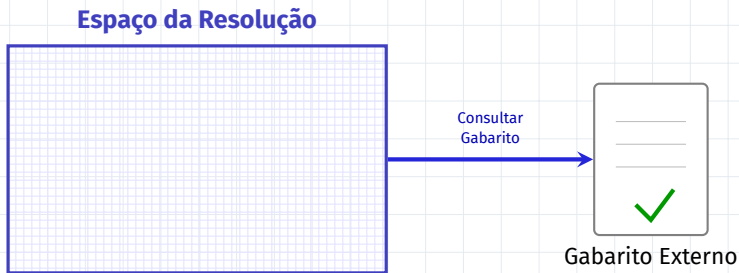
Consultar  
Gabarito



Gabarito Externo

**Nota:** A resolução deste exercício foi movida para o documento com as resoluções, para incentivar a prática.

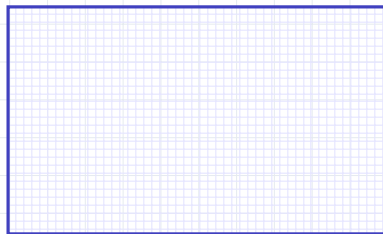




**Nota:** A resolução deste exercício foi movida para o documento com as resoluções, para incentivar a prática.



## Espaço da Resolução



Consultar  
Gabarito



Gabarito Externo

**Nota:** A resolução deste exercício foi movida para o documento com as resoluções, para incentivar a prática.



# Concluindo...

---



| Elemento UML           | Mapeamento Python                            | Exemplo EE                   |
|------------------------|----------------------------------------------|------------------------------|
| Objeto / Linha de vida | Instância de classe                          | <code>scada = SCADA()</code> |
| Mensagem síncrona      | Chamada de método                            | <code>clp.read(addr)</code>  |
| Mensagem assíncrona    | Publish / callback                           | <code>mqtt.publish()</code>  |
| Retorno                | <code>return</code>                          | <code>return valor</code>    |
| Auto-mensagem          | <code>self._metodo()</code>                  | <code>self._validar()</code> |
| Criação / Destruição   | <code>__init__</code> / <code>close()</code> | <code>Sessao()</code>        |
| <b>alt</b>             | <code>if/elif/else</code>                    | Alarme vs registro           |
| <b>loop</b>            | <code>while</code> / <code>for</code>        | Polling de sensores          |
| <b>opt</b>             | <code>if cond: (sem else)</code>             | Log opcional                 |
| <b>break</b>           | <code>raise</code> / <code>break</code>      | Timeout de IED               |
| <b>par</b>             | <code>ThreadPoolExecutor</code>              | Leitura paralela             |

## FAÇA:

- ▶ Um diagrama por cenário do caso de uso
- ▶ Numere as mensagens sequencialmente
- ▶ Indique o tipo (síncrona vs assíncrona)
- ▶ Use fragmentos para fluxos condicionais
- ▶ Mostre retornos para mensagens síncronas
- ▶ Identifique auto-mensagens (processamento interno)

## EVITE:

- ▶ Mais de 8–10 objetos em um diagrama
- ▶ Misturar cenário principal e alternativo
- ▶ Omitir retornos de chamadas síncronas
- ▶ Fragmentos aninhados em mais de 2 níveis
- ▶ Mensagens sem nome ou com nomes vagos
- ▶ Colocar lógica de UI nos objetos de domínio

## Armadilhas frequentes ao modelar sequências:

### ✘ Erro

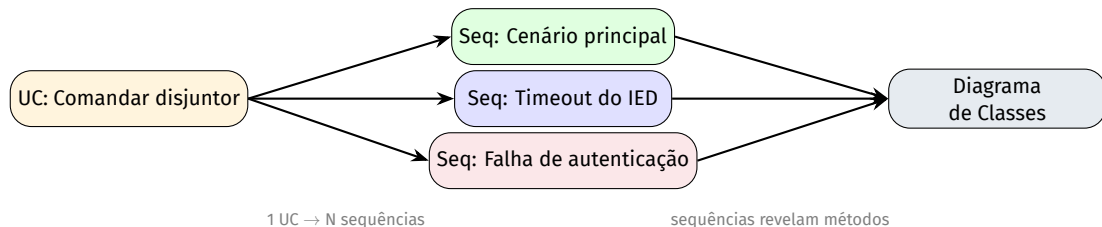
- ▶ Seta síncrona para MQTT (que é assíncrono)
- ▶ Retorno sem mensagem síncrona correspondente
- ▶ Objeto que recebe mensagem sem barra de ativação
- ▶ Mensagem “processa dados” (vaga demais)
- ▶ Fragmento *alt* sem condição de guarda

### ✔ Correto

- ▶ Seta tracejada para MQTT (assíncrona)
- ▶ Cada chamada síncrona tem seu retorno
- ▶ Barra ativa durante processamento
- ▶ Mensagem `ler_analogica(AI-01)` (específica)
- ▶ `alt [corrente > 100A] / [else]`



Cada caso de uso gera um ou mais diagramas de sequência:

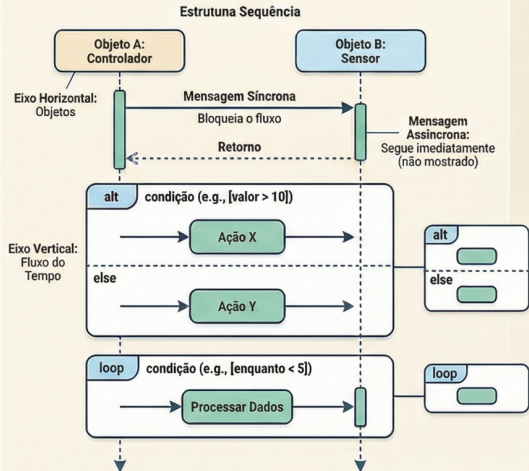


**Regra prática:** Para cada caso de uso, crie pelo menos: **1 seqüência para o cenário principal** (happy path) e **1 seqüência para cada cenário de erro** relevante (timeout, rejeição, falha de autenticação).

# Dominando Diagramas de Sequência: Da UML ao Código Python

O Diagrama de Sequência modela a interação dinâmica entre objetos ao longo do tempo. Este guia foca na notação visual e na tradução direta para estruturas de código Python.

## Fundamentos e Notação Visual

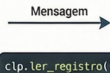


## Mapeamento UML para Python



### Data Table

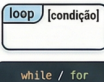
#### Mensagem Síncrona



#### Fragmento alt



#### Fragmento loop





**Prof. Edson José Pacheco**

[pacheco@ufpr.br](mailto:pacheco@ufpr.br)

UFPR

UNIVERSIDADE FEDERAL DO PARANÁ